

GitHub Copilot SDK

From chat to controlled engineering workflows

What we discovered • How it works • Developer and QA use cases • What we would build next

THE CORE IDEA

Copilot becomes a programmable runtime inside our application—not just a chat window.

What we will cover

A practical knowledge transfer—not a feature catalog.

01 Why

The gap between chat assistance and repeatable engineering workflows

02 How

Client, sessions, models, tools, skills, agents, and events

03 Use cases

Where developers and QA teams can get leverage

04 Prototype

Issue triage → code → Jenkins → Serenity → retry or close

05 Reality check

Security boundaries, version compatibility, and remaining work

Why use an SDK instead of only chat?

The value is repeatability, integration, and control.

Chat experience

- Human starts every interaction
- Context is often conversational
- Output may be free-form
- Useful for exploration and assistance



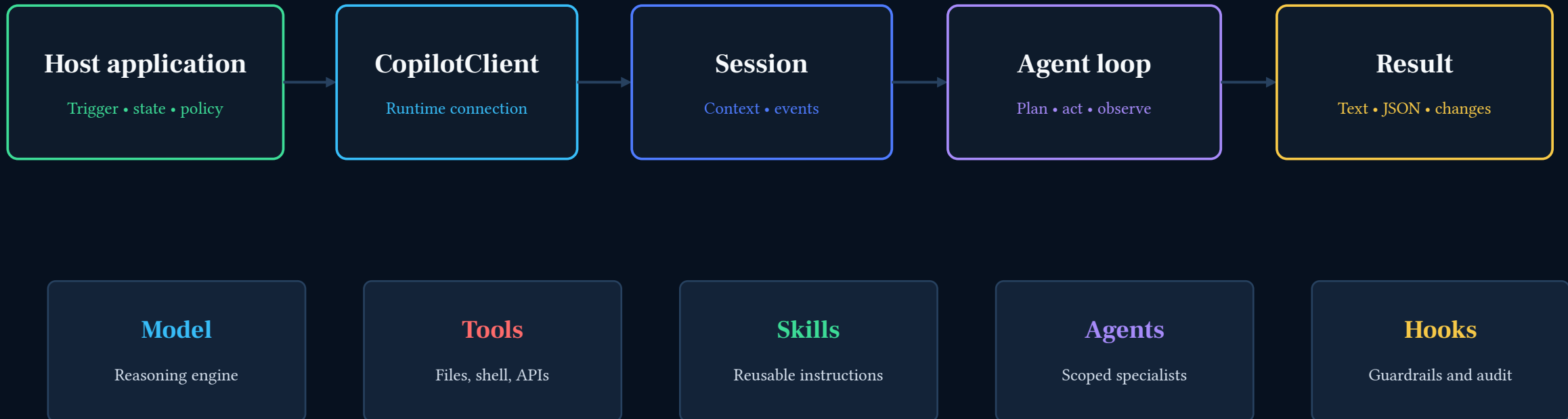
Programmable agent runtime

- Application creates and resumes sessions
- Code controls available tools, permissions, and budgets
- Skills and custom agents package domain behavior
- Events make execution observable
- Structured contracts connect agent stages to CI and business systems

The application owns the workflow. Copilot supplies reasoning and tool use inside controlled boundaries.

How it works: the mental model

A host application surrounds each Copilot session with policy and state.



Events stream what happened back to the host, where we can persist status, render progress, or enforce a gate.

Skills and agents: related, but different

Progressive disclosure is useful only when we understand the loading rules.

Skills

Reusable instruction modules

Good for:

- Dynatrace query procedures
- Jenkins invocation rules
- Test-report interpretation
- Organization-specific playbooks

Custom agents

Named specialists with their own prompt and tool scope

Good for:

- Read-only repository research
- Code implementation
- Security review
- QA analysis

Important rule

Custom agents do not automatically inherit skills from the parent.

Preload named skills on the agent or make them available through the session's configured skill directories.

Discovery answers “what is available?” Invocation answers “what was actually used?”

What we built during Innovation Week

A runnable learning repository—not a production automation platform.

Model catalog

Inspect account-available models and fall back safely.

Skill discovery

List personal and project skills; inspect actual invocation.

Custom agent

Create a read-only repository researcher with scoped behavior.

Issue triage

Read labeled issues with the GitHub CLI and return validated JSON.

Agent Factory

Coordinate bounded parallel analysts and a synthesis stage.

Fresh handoff

Validate JSON, end one session, and start the next with only trusted data.

Every example is intentionally small so the team can see one SDK concept at a time.

Developer use cases

Best fit: bounded work with clear ownership, evidence, and validation.

Issue readiness

Read-only

Rank eligible work, identify missing acceptance criteria, and recommend the safest starting point.

Repository research

Read-only

Trace code paths, dependencies, and likely change surfaces before editing.

Targeted implementation

Write

Create a worktree, make scoped changes, and return changed files plus test evidence.

Parallel review

Parallel

Fan out security, architecture, testability, and documentation reviews; synthesize findings.

Migration support

Fleet

Divide independent packages or files among workers and verify the combined result.

Release handoff

Structured

Produce a typed summary for CI, release notes, or a human approval gate.

QA use cases

The strongest QA value is evidence synthesis and coverage expansion—not autonomous pass/fail authority.

01 Requirements

Turn acceptance criteria into risks, scenarios, and traceability.

02 Test design

Generate positive, negative, boundary, and data-variation cases.

03 Execution support

Prepare environment checks, targeted suites, and reproducible commands.

04 Failure triage

Correlate changed files, logs, stack traces, and prior attempts.

05 Report analysis

Summarize Serenity/JUnit evidence and highlight regression clusters.

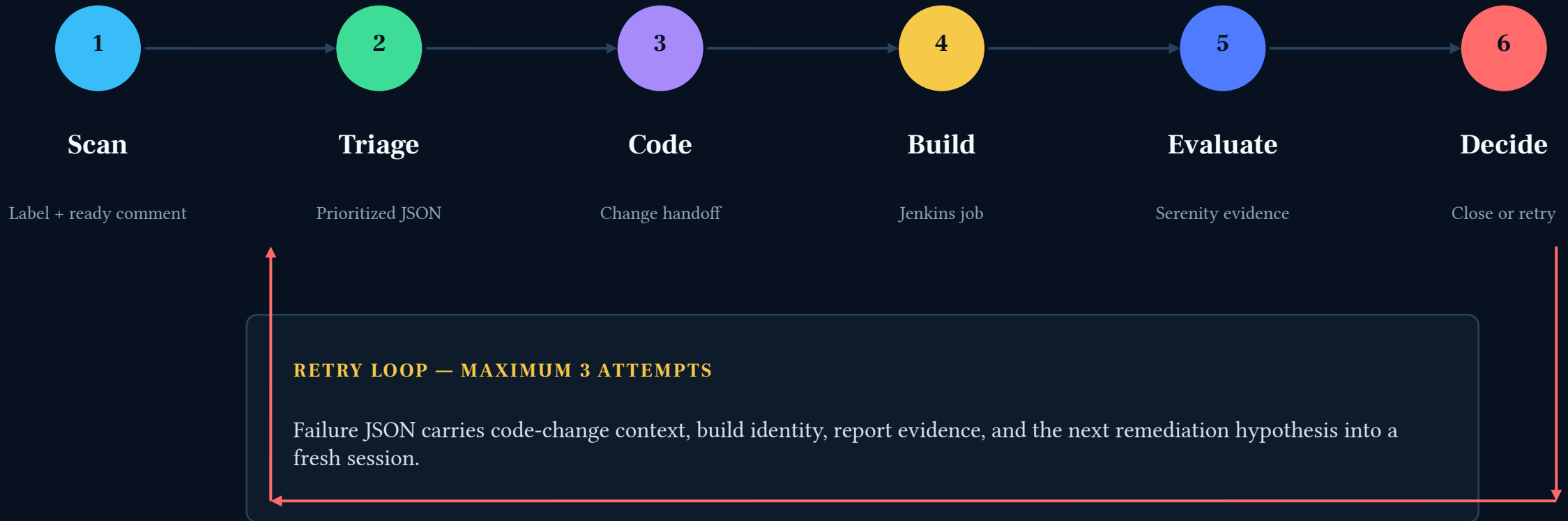
06 Quality gate

Recommend action while deterministic systems retain pass/fail authority.

QA remains the owner of quality strategy; the agent accelerates analysis and broadens coverage.

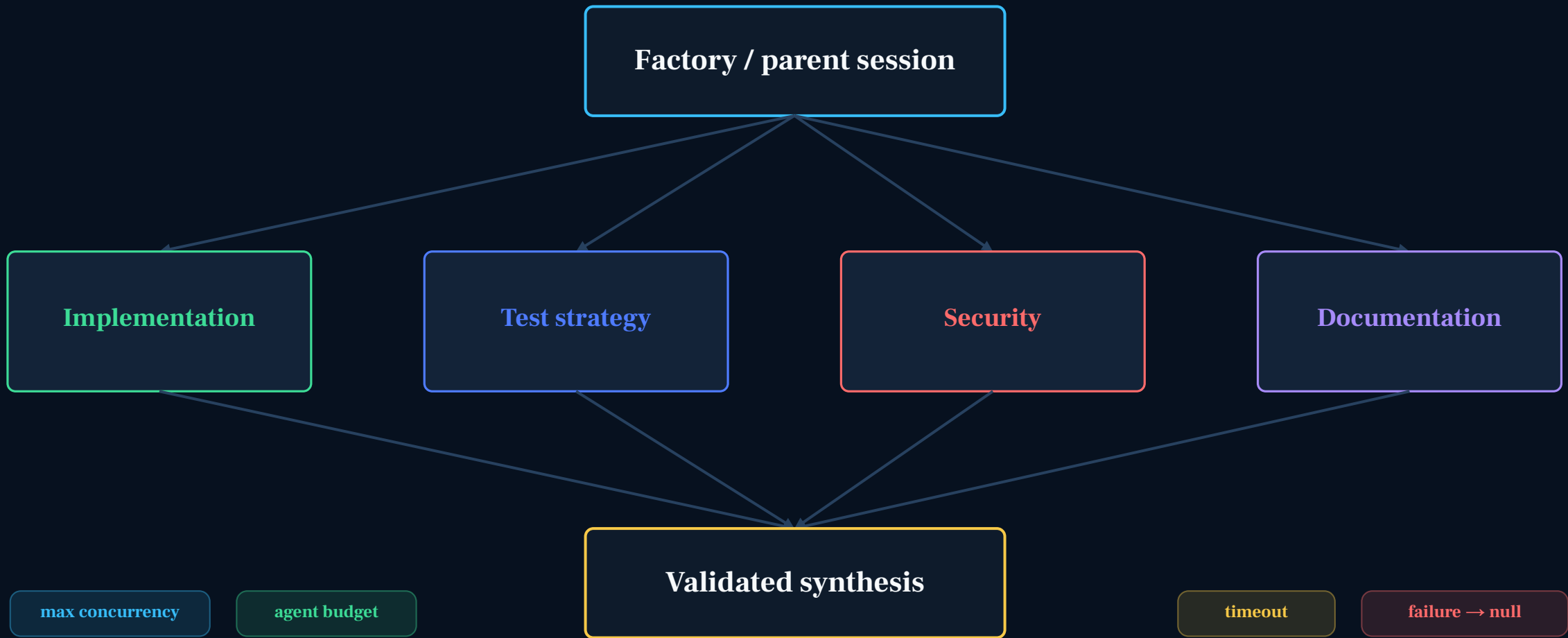
The end-to-end workflow we designed

Short-lived agent sessions inside a durable, deterministic controller.



Fan-out / fan-in for independent analysis

Parallelism helps when workers do not contend for the same state.



What the application must control

An agent can recommend actions; trusted code authorizes and verifies them.

Application / controller

- ✓ Verify webhook actor and repository
- ✓ Claim work idempotently
- ✓ Restrict tools and credentials
- ✓ Poll Jenkins and persist state
- ✓ Validate JSON contracts
- ✓ Verify commit + build + report before closure

Copilot session

- ✓ Interpret goals and evidence
- ✓ Select or invoke appropriate skills
- ✓ Research code and propose changes
- ✓ Use allowed tools with permission checks
- ✓ Explain failures and remediation options
- ✗ Never becomes credential or completion authority

What we learned

The prototype answered architectural questions—and exposed integration risks.

01 Sessions are boundaries

Fresh sessions reduce accidental context carryover, but JSON still needs validation.

02 Discovery ≠ invocation

A visible skill is not proof the runtime actually selected and followed it.

03 Models are account-specific

List what is available; never hard-code an assumed model without fallback.

04 Extensions are experimental

Agent Factory and fleet behavior can depend on matched SDK and runtime versions.

05 Polling is ordinary software

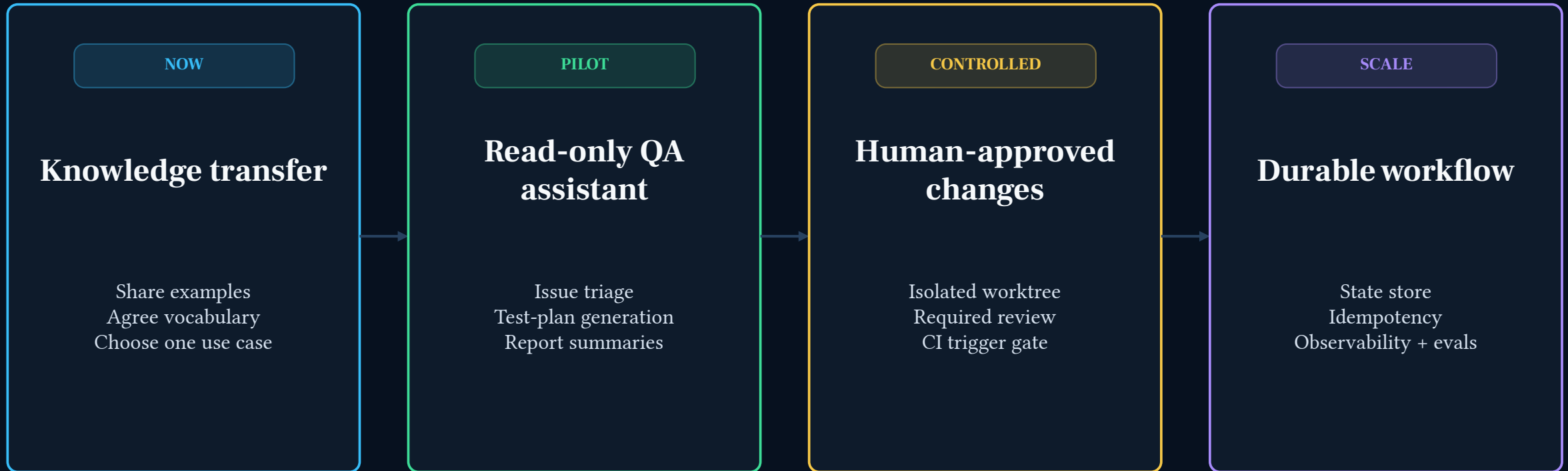
Long waits, retries, idempotency, and state belong outside the model loop.

06 Evidence beats confidence

Tests, build identity, and parsed reports—not fluent text—authorize completion.

A practical adoption path

Move from read-only evidence to bounded automation in stages.



Recommended first pilot: read-only issue readiness + QA test-plan generation with human review.

TAKEAWAY

Copilot SDK is most valuable when we combine agent reasoning with ordinary engineering discipline.

Reasoning

Copilot sessions, skills, agents, and parallel workers

Control

Permissions, state, retries, schemas, and deterministic checks

Confidence

Evaluation cases, CI evidence, observability, and human gates

Discussion

Which developer or QA workflow is bounded enough for our first pilot?

Sources and demo repository

Primary documentation used for the current capability statements.

[GitHub Copilot SDK](#)

github.com/github/copilot-sdk

[SDK feature guides](#)

docs.github.com/en/copilot/how-tos/copilot-sdk/features

[Custom agents and skill loading](#)

docs.github.com/en/copilot/how-tos/copilot-sdk/features/custom-agents

[Fleet mode](#)

docs.github.com/en/copilot/how-tos/copilot-sdk/features/fleet-mode

[Hooks and permission controls](#)

github.com/github/copilot-sdk/blob/main/docs/features/hooks.md

[Innovation Week examples](#)

github.com/blockvoltr7/copilot-sdk-workflow-examples

Capability details are evolving. Pin SDK/runtime versions and revalidate experimental features before production use.